

Course Assignment Report

Algorithms 2.2

Maksym Kotsiubailo
Student ID: 424602

June 16, 2026

Contents

1	Introduction	2
2	Methodology	2
2.1	Environment	2
2.2	Input Data	2
2.3	Theoretical Expectations	3
3	Task 1A: Spell Checker ("Me spell rite")	3
3.1	Structures Compared	3
3.2	Measured Results	3
4	Task 1B: Triwizard Tournament	5
5	Task 1C: Aunt's Namesday	5
6	Task 2: Full House (Hash Tables)	5
6.1	Setup	5
6.2	Selected Numerical Points	5
7	Task 3: Dictionary Races	8
7.1	Compared Structures	8
7.2	Build Time and Lookup Time	8
7.3	Build Memory	8
8	Validity Notes	10
9	Conclusion	10

1 Introduction

This report summarizes the implementation and evaluation of data-structure-based algorithms in the `Algorithms2.2` project (C#, .NET 8). The document covers:

- spell checking with multiple dictionary structures,
- graph tasks (shortest-path race and bipartite seating),
- hash table behavior under changing load factor,
- dictionary build/lookup races including memory analysis.

All figures included in this report are taken from project-generated benchmark outputs.

2 Methodology

2.1 Environment

Benchmarks were executed with BenchmarkDotNet v0.15.8 on Windows 11 with .NET 8.0 runtime. The benchmark host reports:

- CPU: AMD Ryzen 5 4600H (6 physical / 12 logical cores),
- Runtime: .NET 8.0.24, X64 RyuJIT,
- SDK: .NET SDK 10.0.300.

Plotting was performed with Python and Matplotlib using the benchmark CSV artifacts.

2.2 Input Data

The project uses dictionary and text files stored under `data/`. Table 1 lists the primary datasets used in the tasks and benchmark setup.

Table 1: Input datasets used by the assignment tasks

Dataset	Lines	Size (bytes)
<code>english_words.txt</code>	109,582	1,154,335
<code>english_names.txt</code>	99,154	1,037,854
<code>The_Gift_of_the_Magi.txt</code>	45	11,335
<code>little_prince__with_errors.txt</code>	1,163	96,121
<code>war_and_peace.txt</code>	50,886	3,266,164
<code>party.txt</code>	6	86

2.3 Theoretical Expectations

Expected complexity trends used to interpret benchmark behavior:

- BBST build: approximately $O(n \log n)$,
- Trie and TST build: approximately $O(nL)$, where L is average key length,
- Hash-based lookup: expected near $O(1)$ average,
- Tree lookup: approximately $O(\log n)$,
- Trie lookup: approximately $O(L)$.

3 Task 1A: Spell Checker ("Me spell rite")

3.1 Structures Compared

The spell-checking pipeline compares dictionary representations:

- Naive linear scan,
- HashMap (hash-based set lookup),
- BBST (red-black tree),
- Ordinary trie.

3.2 Measured Results

Table 2 contains benchmark values exported by BenchmarkDotNet for the current run artifacts.

Table 2: Spell checker benchmark summary

Method	Mean time (ms)	Allocated (KB)
BenchmarkNaive	8099.64	725.98
BenchmarkHashMap	16.46	5980.40
BenchmarkBBST	73.94	10931.39
BenchmarkTrie	204.17	69177.37
BenchmarkHashMapBuild	15.65	5896.86
BenchmarkBBSTBuild	62.27	10847.56
BenchmarkTrieBuild	203.92	69009.35

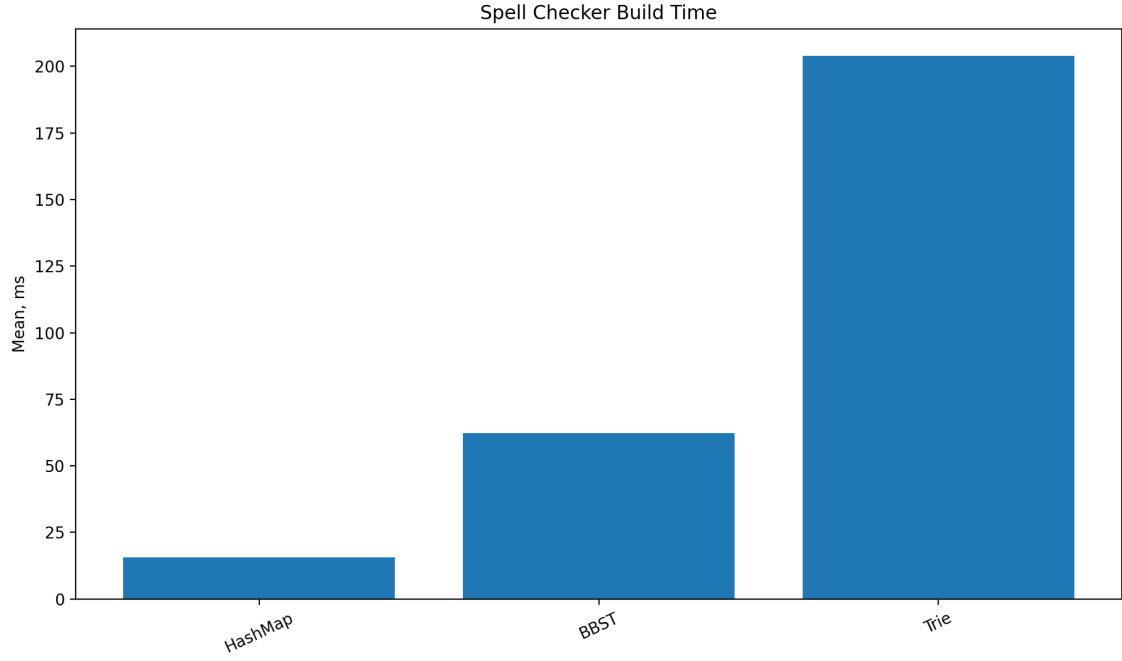


Figure 1: Spell checker dictionary build time

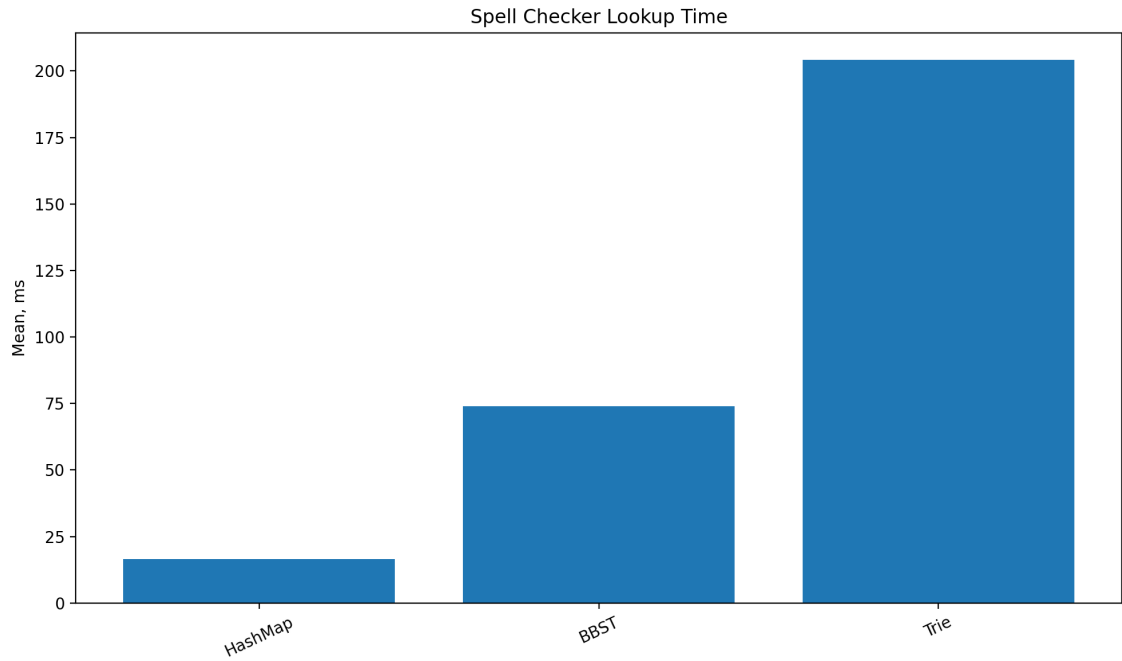


Figure 2: Spell checker lookup time

Interpretation. The hash-based variant is the fastest among measured structures for this workload. BBST is slower but still substantially better than linear scanning. Trie has the highest memory footprint and the highest measured times in this specific setup, which indicates that implementation constants and allocation behavior dominate asymptotic expectations for the tested input scale.

4 Task 1B: Triwizard Tournament

The labyrinth race uses a single BFS traversal on a 2D grid with four-neighbor expansion. Distances from finish cell to all reachable cells are computed once, then each wizard's travel time is calculated as:

$$t_i = \frac{d_i}{v_i}$$

where d_i is shortest path length in grid edges and v_i is wizard speed.

This approach is optimal for unweighted grids and avoids repeated shortest-path computations for each participant.

5 Task 1C: Aunt's Namesday

The seating task is solved as a bipartite graph check. Guests are vertices and conflict pairs are edges. A non-recursive DFS assigns labels $\{+1, -1\}$ (table 1/table 2). If an edge connects vertices with the same label, seating is impossible. Otherwise, the labeling gives a valid two-table partition.

6 Task 2: Full House (Hash Tables)

6.1 Setup

Three custom hash table implementations are benchmarked:

- separate chaining,
- linear probing,
- double hashing.

Key benchmark parameters from source configuration:

- table capacity: 15,013,
- load factor range: 0.05 to 0.95 (step 0.025),
- operations: successful search, failed search, insert.

6.2 Selected Numerical Points

Table 3 presents selected mean values from the BenchmarkDotNet markdown report at representative load factors.

Table 3: Selected hash table means (microseconds)

Method	LF 0.10	LF 0.50	LF 0.70	LF 0.85	LF 0.95
SearchSuccess Chaining	4.718	4.656	4.700	4.308	4.412
SearchSuccess Linear	3.092	3.029	2.888	2.394	2.586
SearchSuccess Double	3.352	2.461	3.032	2.490	2.285
SearchFail Chaining	2.587	2.263	4.486	2.543	4.688
SearchFail Linear	2.340	2.187	2.370	3.261	3.332
SearchFail Double	3.954	4.087	3.288	3.648	3.598
Insert Chaining	3.651	3.227	3.142	2.823	2.407
Insert Linear	3.756	3.443	3.316	3.154	2.827
Insert Double	3.419	3.364	2.962	3.018	2.371

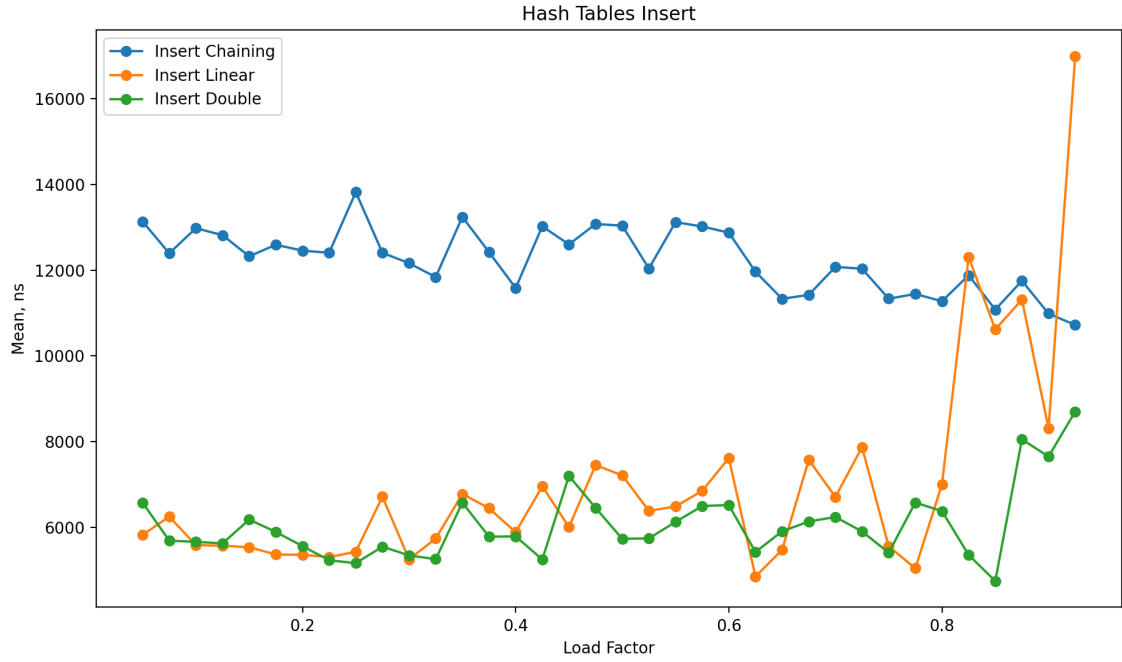


Figure 3: Hash table insert performance across load factors

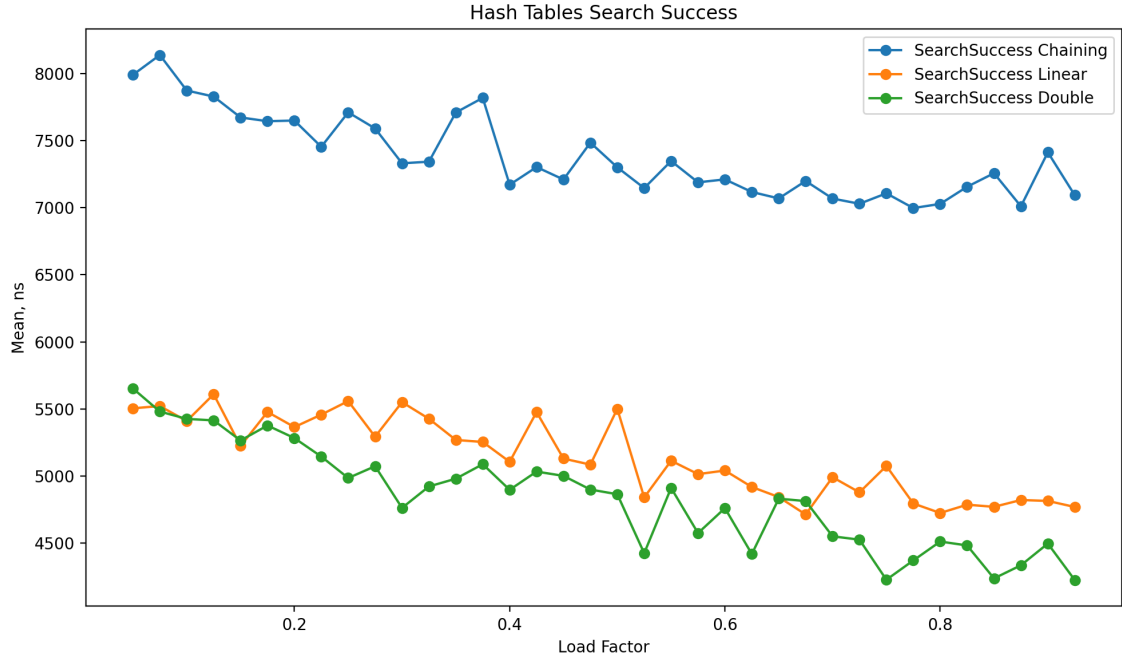


Figure 4: Hash table successful search across load factors

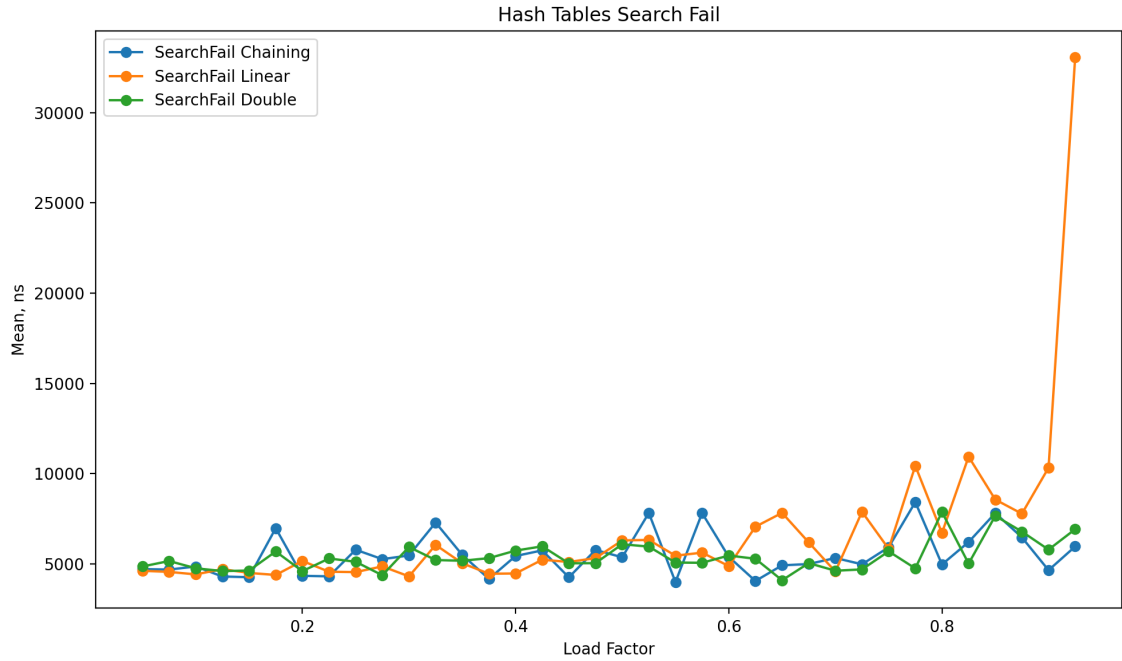


Figure 5: Hash table failed search across load factors

Interpretation. Successful lookups remain relatively stable across load factors for all methods in this run, while failed lookup and insert show higher variance near high occupancy. Linear probing and double hashing exhibit sensitivity spikes close to heavy load, consistent with clustering and probe-chain effects.

7 Task 3: Dictionary Races

7.1 Compared Structures

The benchmark compares:

- red-black tree,
- ordinary trie,
- ternary search tree,
- double-array trie (DAT).

Dictionary size parameter is benchmarked at 2,000; 5,000; 10,000; 20,000; and 30,000 keys.

7.2 Build Time and Lookup Time

Table 4 summarizes means extracted from `Algorithms.BenchmarkDictionaryRaces-report.csv`.

Table 4: Dictionary races: mean time (ms)

Size	Build				Lookup			
	RBTree	Trie	TST	DAT	RBTree	Trie	TST	DAT
2,000	0.8048	0.5409	0.9695	239.5860	2.1006	0.9224	2.4248	2.4142
5,000	2.2170	1.4932	2.6882	1414.3338	2.5873	0.9348	2.4653	2.4331
10,000	4.6755	3.1671	5.2790	5467.7353	2.8237	0.9658	2.5815	2.4603
20,000	9.8919	6.8769	10.9867	18365.2912	3.2091	1.0258	2.6736	2.4602
30,000	15.3000	11.0791	17.6225	40825.0632	3.5915	1.0920	2.8802	2.5906

7.3 Build Memory

Table 5 reports allocated bytes for build benchmarks.

Table 5: Dictionary races: allocated memory during build (bytes)

Size	RBTree	Trie	TST	DAT
2,000	96,032	1,219,176	550,480	306,145,264
5,000	240,036	3,108,888	1,383,344	2,009,425,296
10,000	480,032	6,059,616	2,719,888	7,448,836,048
20,000	960,032	11,633,184	5,350,840	26,783,891,888
30,000	1,440,032	17,408,976	8,029,944	59,982,400,048

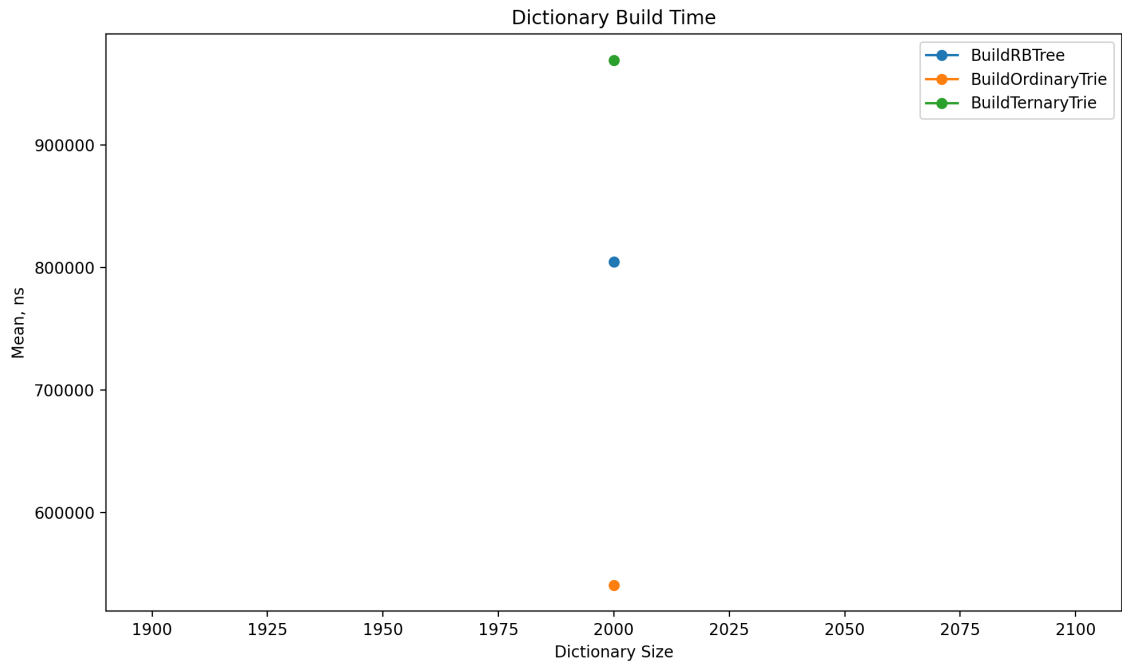


Figure 6: Dictionary races: build time vs dictionary size

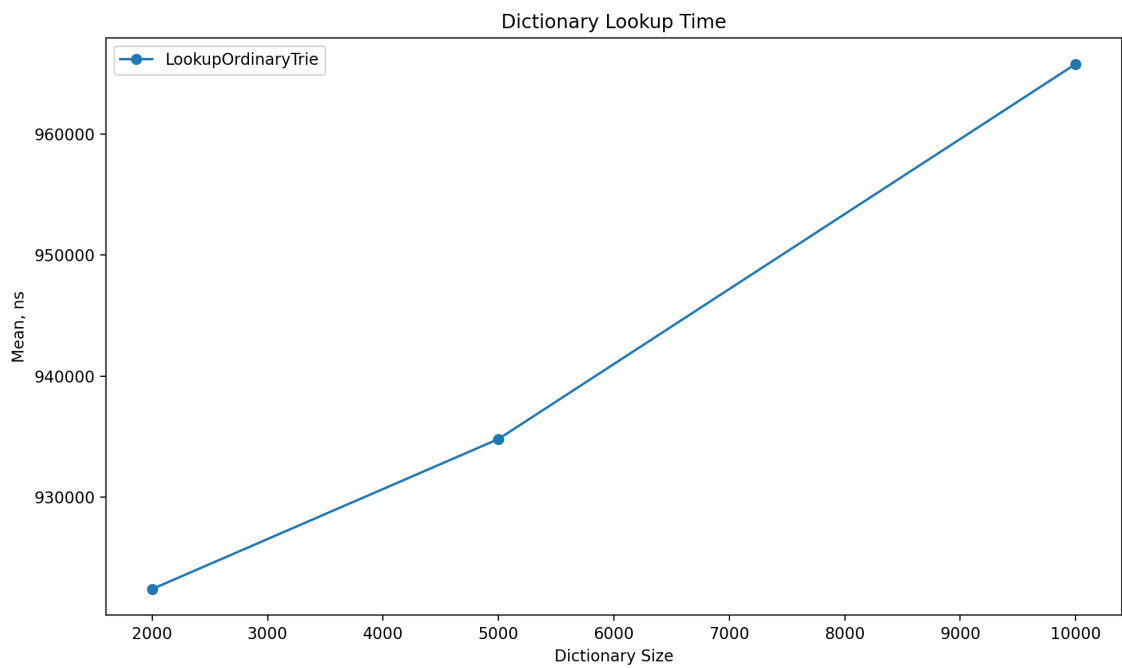


Figure 7: Dictionary races: lookup time vs dictionary size

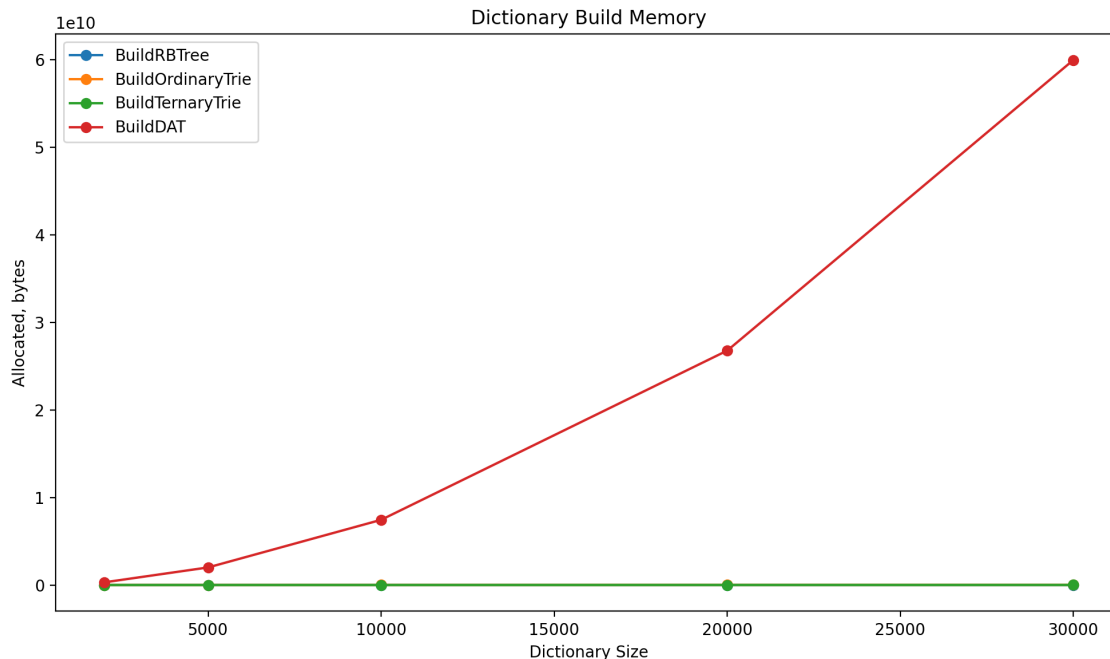


Figure 8: Dictionary races: build memory vs dictionary size

Interpretation. RBTrie, ordinary trie, and TST show predictable growth with dictionary size. DAT exhibits extreme growth in both build time and memory consumption in this implementation. Lookup latency remains best for ordinary trie and close between TST/DAT, while RBTrie lookup grows with size as expected.

8 Validity Notes

The presented conclusions are valid for this implementation and these datasets. Constant factors, memory allocation strategy, and preprocessing details can dominate asymptotic behavior in practical runs. For strict reproducibility, keep runtime, benchmark config, and dataset versions fixed.

9 Conclusion

The experiments demonstrate that implementation details are critical in algorithm engineering:

- For spell checking, hash-based lookup is the strongest practical baseline in this project.
- For hash tables, all methods remain competitive at moderate load, with increased instability near high occupancy.
- For dictionary races, ordinary trie provides the best lookup latency in this run, while DAT is currently not competitive for build cost and memory.
- BFS and bipartite DFS formulations correctly model Tasks 1B and 1C and give efficient graph-based solutions.